Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul *Ingineria Software*

Report on laboratory work no. 7

Course: Operating Systems

Topic: Three Small Programs: Processes, Redirection, a Pipe

Done by: st. gr. FAF-241, Lungu Ilie

Checked by: *Relitman Patricia*

Chişinău – 2026

Contents

Purpose of the work **2**

1 PART 1 – FORK + EXEC + WAIT **2**
1.1 Observe 3

2 PART 2 – A ZOMBIE, ON PURPOSE **3**
2.1 Observe 4

3 PART 3 – REDIRECTION: LS -L > OUT.TXT **4**
3.1 Observe 5

4 PART 4 – A PIPE: LS /ETC | WC -L **5**
4.1 Observe 6
4.2 Optional: three stages 6

Conclusions **7**

PURPOSE OF THE WORK

The purpose of this laboratory work is to run the answers to three questions of the lecture: how a process is born, where `printf` writes, and how `ls | wc -l` works. Three short C programs are compiled and run, one line is changed in each of them, and the result is observed: `run.c` does `fork + exec + wait` (what the shell does for `ls -l`), `zombie.c` makes a zombie process on purpose, `redirect.c` does `ls -l > out.txt` with `close` and `open`, and `pipeline.c` builds `ls /etc | wc -l` with one `pipe` and two `fork`s. The optional program `pipeline3.c` (three stages) was also done. All the programs were compiled with `gcc -Wall -Wextra` (GCC 13.3, no warnings) and run on **Ubuntu 24.04 LTS** (a Debian-family Linux) in the `bash` shell as a normal user, in the folder `~/os-lab7`. The answers to all *Observe* boxes are given in the corresponding parts, with the commands and the real outputs.

```
$ mkdir -p ~/os-lab7 && cd ~/os-lab7
$ gcc --version | head -1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
```

Figure 0.1 – Preparation

1 PART 1 – FORK + EXEC + WAIT

The program `run.c` is the code of the assignment, without changes. It does the three steps of the shell for the command `ls -l`: `fork` (copy myself), `execlp` (the copy becomes `ls`) and `waitpid` (the original waits).

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command: ls -l */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                 /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                     /* only the child runs this */
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);   /* step 2: become ls */
        perror("exec");                 /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);           /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Figure 1.1 – run.c

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 215
child: my PID is 216, my parent is 215
total 44
-rwxr-xr-x 1 ilie root 16344 Oct 26 20:01 redirect
-rw-r--r-- 1 ilie root   691 Oct 26 20:01 redirect.c
-rwxr-xr-x 1 ilie root 16400 Oct 26 20:01 run
-rw-r--r-- 1 ilie root   871 Oct 26 20:01 run.c
parent: child 216 ended with status 0
```

Figure 1.2 – Compile and run (the folder contained only the files shown)

The order of the lines is the one of the assignment. In the next two experiments the line with `execlp` was replaced, as asked, by the call of `date`, and then by the call of a program that does not exist (each variant is a separate file, `run_date.c` and `run_nosuch.c`).

```
$ gcc -Wall -Wextra -o run_date run_date.c && ./run_date
parent: my PID is 159
child: my PID is 160, my parent is 159
Mon Oct 26 20:00:31 UTC 2026
parent: child 160 ended with status 0
$ gcc -Wall -Wextra -o run_nosuch run_nosuch.c && ./run_nosuch
parent: my PID is 186
child: my PID is 187, my parent is 186
exec: No such file or directory
parent: child 187 ended with status 127
```

Figure 1.3 – The two variants of the execlp line

1.1 Observe

1. **Why `perror("exec")` is printed only with `nosuchprog`.** When `execlp` succeeds, it does not return: it replaces the whole program of the process (code, data, stack) with the new program, so the old code, including the line `perror("exec")`, does not exist any more. The child becomes `ls` (or `date`) and ends when this program ends. `execlp` returns, with -1, only when it fails; for `nosuchprog` no such file is found in any directory of `PATH`, the program of the child is not replaced, and the next lines run: `perror` prints `exec: No such file or directory` and `exit(127)` follows. (The last run was made with a standard `PATH`; if one directory of `PATH` cannot be read by the user, `execlp` may report `Permission denied` instead, which was seen once in another shell.)

2. **The number after "my parent".** It is **215** in Figure 1.2: the PID of the *parent* process, the one that printed "parent: my PID is 215". `getppid()` in the child returns the PID of the process that called `fork`. The PIDs of the child (216) is the value that `fork` returns in the parent, and the parent prints it again in the last line.

3. **The status.** With `ls` (and with `date`) the parent sees **0**, the normal exit code of a program that worked. With `nosuchprog` it sees **127**: this is the value of the `exit(127)` of the child after the failed `exec`, the code that the shell also uses for "command not found".

2 PART 2 – A ZOMBIE, ON PURPOSE

The file `zombie.c` is the copy of `run.c` (the version with `ls`) with one new line, `sleep(20);`, just before `waitpid`:

```
    int status;
    sleep(20);                          /* the new line */
    waitpid(pid, &status, 0);           /* step 3: wait for the child */
```

Figure 2.1 – The changed part of zombie.c

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
$ sleep 2; ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
  185   184 S    bash /tmp/l7b.sh
  188   185 S    ./zombie
  190   188 Z    [ls] <defunct>
$ sleep 20; ps -o pid,ppid,stat,cmd
```

```
  PID  PPID STAT CMD
  185   184 S    bash /tmp/l7b.sh
```

Figure 2.2 – The process table while the parent sleeps, and after 20 seconds

The lines are those of the experiment (the `bash` line is the shell that ran the commands; here it was a script, in a terminal it is your login shell, and the `ps` line itself is left out because the command was restricted to the related processes).

2.1 Observe

1. **The letter of `ls`.** It is **Z**: *zombie* (the command is shown as `[ls] <defunct>`). The child has already ended, but its parent (`./zombie`, PID 188, the PPID of the line) has not yet called `wait`, so the kernel keeps an entry in the process table with the exit status until the parent asks for it. The zombie uses no CPU and no memory, but it holds a PID and the status.

2. **After 20 seconds.** No, the line `[ls] <defunct>` is not there any more (and `./zombie` also ended). The line that made it disappear is `waitpid(pid, &status, 0);`: it *reaps* the child, i.e. returns its status to the parent and frees the entry. The new line `sleep(20)` only delayed this call by 20 seconds, and this delay is the time during which the zombie was visible.

3 PART 3 – REDIRECTION: LS -L > OUT.TXT

The program `redirect.c` does what the shell does for `ls -l > out.txt`: in the child it closes the descriptor 1, opens the file (which gets the smallest free number), and then runs `ls`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls -l > out.txt */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                       /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);   /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Figure 3.1 – redirect.c

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 44
-rw-r--r-- 1 ilie ilie     0 Oct 26 20:01 out.txt
-rwxr-xr-x 1 ilie root 16344 Oct 26 20:01 redirect
```

Figure 3.2 – Compile, run, and look inside the file

3.1 Observe

1. **The number given by `open`. 1** ("open gave descriptor 1"). `open` always returns the *smallest number that is free*. In the child the descriptors 0 (input) and 2 (errors) are still open and the program has just closed 1, so 1 is the smallest free number. This is the whole trick of redirection: the file takes exactly the place of the standard output. (The message is written with `fprintf(stderr, ...)` because `stdout` is the file now; this is why the message is seen on the screen.)

2. **Why the listing is not on the screen.** `ls` writes its result to the descriptor 1, as always, and it does not know where it leads. The table of open descriptors is kept by `exec`, so the new program `ls` inherits it, and there the descriptor 1 points to `out.txt`, so the listing goes into the file. The shell does the same before `exec`; the program `ls` is not changed. (A detail visible in Figure 3.2: `out.txt` is in its own listing with the size 0, because `open` created it before `ls` read the directory, and the text was written into it only after the listing was prepared.)

3. **To redirect the errors** (like `2> err.txt`) the descriptor **2** must be closed before `open`, so that the file gets the number 2.

4 PART 4 – A PIPE: LS /ETC | WC -L

The program `pipeline.c` builds the pipeline by hand: one `pipe`, two `fork`s. In the child for `ls` the descriptor 1 becomes the writing end `p[1]`; in the child for `wc` the descriptor 0 becomes the reading end `p[0]`. `dup` copies a descriptor into the smallest free number, exactly as `open`. Every process closes the two original ends after `dup`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls /etc | wc -l */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                  /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                      /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                  /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                      /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                        /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Figure 4.1 – pipeline.c

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
141
$ ls /etc | wc -l
141
```

Figure 4.2 – The program and the real shell give the same number

Then the line `close(p[1]);` of the parent (the one after the comment "the parent uses neither end") was turned into a comment, as asked (file `pipeline_broken.c`), and the program was started in the background. The process table and the descriptors were examined with `ps` and `/proc`:

```
$ gcc -Wall -Wextra -o pipeline_broken pipeline_broken.c
$ ./pipeline_broken &
$ ps -o pid,ppid,stat,cmd --ppid $! -p $!
  PID  PPID STAT CMD
  194   185 S    ./pipeline_broken
  197   194 S    wc -l
$ ls -l /proc/194/fd          # the parent (only the descriptor numbers are shown)
0 -> /dev/null
1 -> /tmp/pb.out
2 -> pipe:[940]
4 -> pipe:[1244]
$ ls -l /proc/197/fd          # wc
0 -> pipe:[1244]
1 -> /tmp/pb.out
2 -> pipe:[940]
```

Figure 4.3 – The broken pipeline does not end

(The output of the program was sent to a file and the shell's own pipe `pipe:[940]` is not related to the experiment.) The processes were stopped with `kill -KILL`; this replaces the `Ctrl-C` of the terminal, which is not available in a script. In a first test the parent only was stopped by `timeout` after 5 seconds, and then the number 141 appeared at the moment of its death: this is discussed in the third answer.

4.1 Observe

1. **The same number?** Yes: `./pipeline` and `ls /etc | wc -l` both printed **141** (Figure 4.2).

2. **Which child never ends.** `wc`. In Figure 4.3 only the parent (`./pipeline_broken`) and `wc -l` (state `S`, sleeping) are left; `ls` is not there, because it wrote its names and ended, and the first `wait` of the parent collected it. The parent waits for `wc` in its second `wait`, so the whole program does not end.

3. **What `wc` is waiting for.** `wc -l` reads its input until the *end of file*, and on a pipe the end of file comes only when *all* the writing ends of the pipe are closed; the parent still keeps the writing end open (the descriptor 4, the same `pipe:[1244]` that is the input of `wc`), so `wc` thinks that more data can still come, and the parent waits for `wc`: a deadlock. The test confirms it: when the parent was killed, its descriptor was closed, `wc` received the end of file and immediately printed 141.

4.2 Optional: three stages

The program `pipeline3.c` does `ls /usr/bin | grep x | wc -l` with two pipes and three children; every child closes all four ends after its `dup`, and so does the parent.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls /usr/bin | grep x | wc -l */
int main(void)
{
    int p1[2], p2[2];
    if (pipe(p1) < 0 || pipe(p2) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                  /* ls: 1 -> p1[1] */
        close(1);
        dup(p1[1]);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("ls", "ls", "/usr/bin", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                  /* grep: 0 <- p1[0], 1 -> p2[1] */
        close(0);
        dup(p1[0]);
        close(1);
        dup(p2[1]);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("grep", "grep", "x", (char *)NULL);
        perror("exec grep");
        exit(127);
    }

    if (fork() == 0) {                  /* wc: 0 <- p2[0] */
        close(0);
        dup(p2[0]);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p1[0]); close(p1[1]);         /* the parent uses none of the ends */
    close(p2[0]); close(p2[1]);
    wait(NULL);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Figure 4.4 – pipeline3.c

```
$ gcc -Wall -Wextra -o pipeline3 pipeline3.c && ./pipeline3
201
$ ls /usr/bin | grep x | wc -l
201
```

Figure 4.5 – Three stages and the real shell

The numbers are equal. If one end were left open in any process, the same deadlock as in Figure 4.3 would appear, with the stage after the open end waiting for an end of file that does not come.

CONCLUSIONS

In this laboratory work the main operations of the shell were done by hand in C. The first program showed the three steps of a command: `fork` makes a copy, a successful `exec` replaces the program and never returns (so the line after it runs only on an error), and `wait`

gives the parent the exit status (0 for `ls`, 127 for a program that was not found). The second program showed that a child which ended stays in the process table as a zombie (state `Z`) until the parent calls `wait`. The third program showed that redirection is only a change of the table of descriptors: `close(1)` and then `open` give the file the number 1, and `ls`, which writes to the descriptor 1, goes into the file without being changed, because `exec` keeps the open descriptors.

The fourth program built `ls /etc | wc -l` from one `pipe`, two `fork`s and `dup`, with the same result (141) as the shell. The broken variant showed why every process must close the ends of the pipe it does not use: the reader gets the end of file only when all the writing ends are closed, and a forgotten `close(p[1])` in the parent made `wc` wait forever. The optional program with three stages gave the same number (201) as the shell and confirmed this rule for two pipes.